APPLICATION WEB TRAITEUR ÉVÉNEMENTIEL

VITE & GOURMAND

Documentation de déploiement

Application Web Traiteur Événementiel

Projet réalisé dans le cadre du titre professionnel

Développeur Web et Web Mobile

Version : 1.0 · Date : Mars 2026 · Projet : Vite & Gourmand

Sommaire

Ce document décrit l'installation, la configuration et le déploiement de l'application web Vite & Gourmand. Il couvre les environnements de développement (Docker Compose) et de production (VPS, Docker, PaaS), ainsi que la sécurité et les opérations de maintenance.

01 Architecture technique

02 Prérequis

03 Installation locale (développement)

04 Configuration

05 Base de données

06 Docker — détail des services

07 Déploiement en production

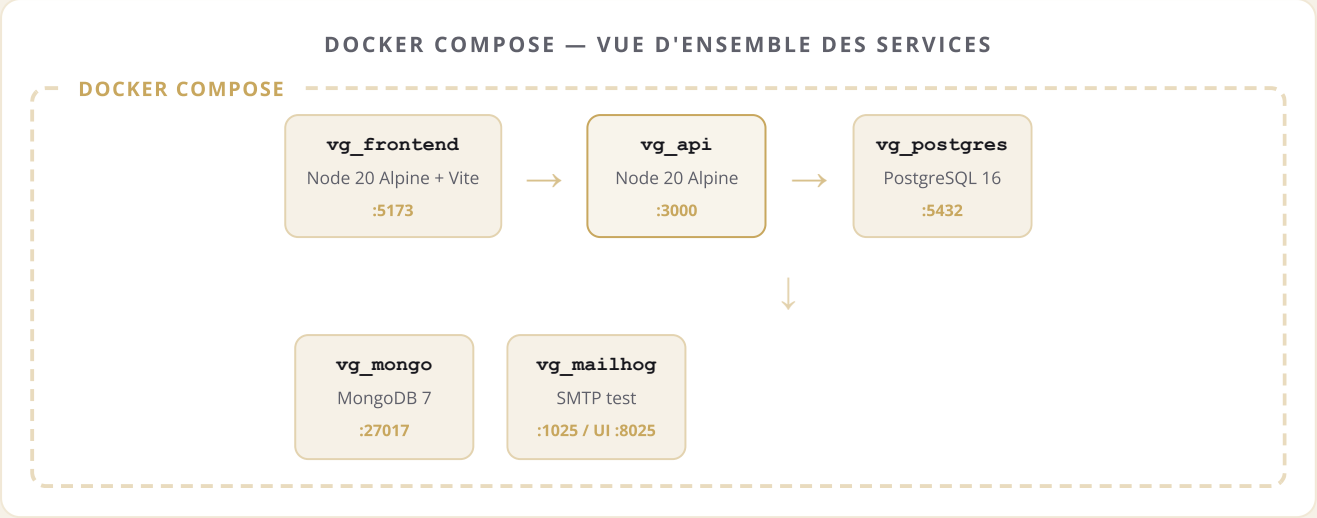
08 Sécurité

09 Tests post-déploiement

10 Maintenance et opérations

Architecture technique

L'application est entièrement conteneurisée via **Docker Compose**. Cinq services collaborent pour former l'environnement complet :



SERVICE	IMAGE	RÔLE	PORT
`vg_frontend`	Node 20 Alpine + Vite	Application React (SPA)	5173
`vg_api`	Node 20 Alpine	API REST Express.js	3000
`vg_postgres`	PostgreSQL 16 Alpine	Base de données relationnelle	5432
`vg_mongo`	MongoDB 7	Analytiques (logs, statistiques)	27017
`vg_mailhog`	MailHog	Interception email en développement	1025 (SMTP), 8025 (UI)

Flux de données :

- Le **frontend** communique avec l'**API** via `VITE_API_URL` (HTTP REST + JWT)
- L'**API** utilise **PostgreSQL** comme base principale (requêtes SQL via `pg`)
- L'**API** utilise **MongoDB** pour les analytiques (Mongoose)
- L'**API** envoie les emails via **SMTP** (MailHog en dev, SMTP réel en prod)

SECTION 02

Prérequis

Développement (avec Docker)

OUTIL	VERSION MINIMALE	VÉRIFICATION
Docker	24+	`docker --version`
Docker Compose	v2+	`docker compose version`
Git	2.x	`git --version`

Développement (sans Docker)

OUTIL	VERSION MINIMALE
Node.js	20 LTS
npm	10+
PostgreSQL	16
MongoDB	7

Node.js	20 LTS
PostgreSQL	16
MongoDB	7
Nginx	dernière stable
Serveur SMTP	(Brevo, SendGrid, etc.)

SECTION 03

Installation locale (développement)

3.1 Cloner le dépôt

```
git clone https://github.com/<votre-org>/vite-gourmand.git
cd vite-gourmand
```

3.2 Configurer l'environnement

```
cp .env.example .env
```

Modifier les valeurs dans `.env` (voir section 4).

3.3 Lancer avec Docker Compose

```
docker compose up --build -d
```

Cette commande construit les images, démarre PostgreSQL et attend le healthcheck, exécute les scripts SQL d'initialisation (schema → seed → migrations), puis démarre MongoDB, MailHog, l'API et le frontend dans le bon ordre.

3.4 Initialiser les données

```
# Seed PostgreSQL (admin + données de démo)
docker exec vg_api npm run seed:postgres

# Seed MongoDB (analytiques)
docker exec vg_api npm run seed:mongo

# Ou les deux en une fois
docker exec vg_api npm run seed
```

3.5 Vérifier le bon fonctionnement

SERVICE	URL	ATTENDU
Frontend	http://localhost:5173	Page d'accueil Vite & Gourmand
API Health	http://localhost:3000/api/health	`{ "status": "ok" }`
MailHog UI	http://localhost:8025	Interface de test email

3.6 Compte admin par défaut

CHAMP	VALEUR
Email	`admin@vitegourmand.fr`
Mot de passe	`Admin123!@#`

Important : Changer ce mot de passe immédiatement en production.

Variables d'environnement à définir dans le fichier `.env` (copié depuis `.env.example`).

PostgreSQL

VARIABLE	DESCRIPTION	EXEMPLE
`POSTGRES_HOST`	Hôte PostgreSQL	`db_sql` (Docker) / `localhost`
`POSTGRES_PORT`	Port	`5432`
`POSTGRES_DB`	Nom de la base	`vitegourmand`
`POSTGRES_USER`	Utilisateur	`vitegourmand`
`POSTGRES_PASSWORD`	Mot de passe	(générer un mot de passe fort)

MongoDB

VARIABLE	DESCRIPTION	EXEMPLE
`MONGO_HOST`	Hôte MongoDB	`db_mongo` (Docker) / `localhost`
`MONGO_PORT`	Port	`27017`
`MONGO_DB`	Nom de la base	`vitegourmand_analytics`
`MONGO_USER`	Utilisateur	`vitegourmand`
`MONGO_PASSWORD`	Mot de passe	(générer un mot de passe fort)

Backend (API)

VARIABLE	DESCRIPTION	EXEMPLE
`NODE_ENV`	Environnement	`development` / `production`
`PORT`	Port de l'API	`3000`
`JWT_SECRET`	Clé secrète JWT (≥32 chars)	(générer avec `openssl rand -hex 32`)
`JWT_REFRESH_SECRET`	Clé secrète refresh token	(générer avec `openssl rand -hex 32`)
`JWT_EXPIRY`	Durée access token	`15m`
`JWT_REFRESH_EXPIRY`	Durée refresh token	`7d`

Email (SMTP)

VARIABLE	DESCRIPTION	DEV	PROD
`SMTP_HOST`	Serveur SMTP	`mailhog`	`smtp.brevo.com`
`SMTP_PORT`	Port SMTP	`1025`	`587`
`SMTP_FROM`	Adresse expéditeur	`contact@vitegourmand.fr`	idem
`SMTP_USER`	Identifiant SMTP	(vide)	(clé API)
`SMTP_PASS`	Mot de passe SMTP	(vide)	(clé API)

Frontend / CORS

VARIABLE	DESCRIPTION	EXEMPLE
`VITE_API_URL`	URL de l'API pour le frontend	`http://localhost:3000/api`
`FRONTEND_URL`	URL du frontend (emails)	`http://localhost:5173`
`FRONTEND_URLS`	Origines CORS autorisées (virgule)	`http://localhost:5173`

`BANK_ACCOUNT_NAME`	Nom du titulaire du compte
`BANK_IBAN`	IBAN pour virement
`BANK_BIC`	Code BIC
`ON_SITE_ADDRESS`	Adresse pour paiement sur place
`ON_SITE_HOURS`	Horaires d'accueil
`ON_SITE_PHONE`	Téléphone

SECTION 05

Base de données

5.1 Schéma PostgreSQL

Les scripts SQL sont exécutés automatiquement au premier démarrage du conteneur PostgreSQL, dans l'ordre :

ORDRE	FICHIER	CONTENU
01	`database/schema.sql`	Tables principales : users, menus, dishes, orders, reviews, etc.
02	`database/seed.sql`	Données de démonstration
03	`database/quotes_migration.sql`	Module devis (quotes, quote_items, quote_options, quote_status_history)
04	`database/payments_migration.sql`	Colonnes paiement sur orders
05	`database/quotes-deposit-request.sql`	Colonnes suivi envoi instructions acompte

5.2 Réinitialiser la base

```
# Arrêter et supprimer le volume PostgreSQL
docker compose down
docker volume rm vite-gourmand_pg_data

# Relancer (réexécute les scripts d'init)
docker compose up -d

# Re-seeder
docker exec vg_api npm run seed
```

5.3 MongoDB (analytiques)

MongoDB stocke les données analytiques (statistiques, logs de visite). Ces données sont non critiques et peuvent être recréées via :

```
docker exec vg_api npm run seed:mongo
```

6.1 Frontend (développement)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY index.html vite.config.js ./
EXPOSE 5173
CMD ["npx", "vite", "--host", "0.0.0.0"]
```

- **Bind mounts :** `src/` et `public/` sont montés en volume → hot reload
- **Volume nommé :** `frontend_node_modules` → isole les `node_modules` du host

6.2 Backend API (développement)

```
FROM node:20-alpine
RUN apk add --no-cache python3 make g++   # pour bcrypt natif
WORKDIR /app
COPY package*.json ./
RUN npm install
EXPOSE 3000
CMD ["npm", "run", "dev"]   # node --watch src/index.js
```

- **Bind mount :** `src/` monté → rechargement automatique (Node `--watch`)
- **Volume nommé :** `api_node_modules` → isole les `node_modules`

6.3 Frontend (production)

```
# Build multi-stage
FROM node:20-alpine AS build
WORKDIR /app
ARG VITE_API_URL
ENV VITE_API_URL=$VITE_API_URL
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

- **Build multi-stage :** image finale ~30 Mo (Nginx Alpine)
- `VITE_API_URL` injecté au build via `ARG`

6.4 Backend API (production)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "src/index.js"]
```

- `npm ci --omit=dev` : n'installe que les dépendances de production
- Pas de `--watch`, exécution directe

PostgreSQL	`pg_isready -U $POSTGRES_USER -d $POSTGRES_DB`	5s (20 retries)
MongoDB	`mongosh --eval "db.adminCommand('ping')"`	5s (10 retries)
API	`HTTP GET /api/health` (via Node)	10s (10 retries)

6.6 Ordre de démarrage

```
vg_postgres ─(healthy)→ vg_api ─(healthy)→ vg_frontend
vg_mongo ──(healthy)→ vg_api
vg_mailhog ─(started)→ vg_api
```

6.7 Volumes persistants

VOLUME	USAGE
`pg_data`	Données PostgreSQL
`mongo_data`	Données MongoDB
`api_node_modules`	Dépendances backend (isolées du host)
`frontend_node_modules`	Dépendances frontend (isolées du host)

Astuce : Après ajout d'une dépendance npm, exécuter :
`docker exec vg_api npm install && docker restart vg_api`
ou pour le frontend : `docker exec vg_frontend npm install && docker restart vg_frontend`

Déploiement en production

7.1 Option A — VPS (serveur dédié)

Architecture cible

```
Internet → Nginx (reverse proxy + SSL)
            ├── /          → fichiers statiques (dist/)
            └── /api/*     → proxy_pass → Node.js :3000
```

1. Provisionner un VPS (Ubuntu 22.04 LTS recommandé, 2 Go RAM minimum)

2. Installer les dépendances

```
# Node.js 20
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# PostgreSQL 16
sudo apt install -y postgresql-16

# Nginx + PM2
sudo apt install -y nginx
sudo npm install -g pm2
```

3. Cloner et configurer

```
git clone <repo> /var/www/vite-gourmand
cd /var/www/vite-gourmand
cp .env.example .env
# Éditer .env avec les valeurs de production
```

4. Installer et builder

```
# Backend
cd backend && npm ci --omit=dev

# Frontend
cd ../frontend && npm ci
VITE_API_URL=https://api.votredomaine.fr/api npm run build
```

5. Configurer Nginx

```
server {
    listen 443 ssl http2;
    server_name votredomaine.fr;
    ssl_certificate     /etc/letsencrypt/live/votredomaine.fr/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/votredomaine.fr/privkey.pem;
    root /var/www/vite-gourmand/frontend/dist;
    index index.html;
    location / { try_files $uri $uri/ /index.html; }
    location /api/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

6. SSL avec Let's Encrypt

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d votredomaine.fr
```

7. Lancer l'API avec PM2

```
cd /var/www/vite-gourmand/backend

pm2 startup
```

7.2 Option B — Docker en production

```
# Build avec les Dockerfiles de production
docker build -t vg-api-prod -f backend/Dockerfile.prod ./backend
docker build -t vg-frontend-prod   --build-arg VITE_API_URL=https://api.votredomaine.fr/api   -f frontend/D

# Lancer avec docker compose (adapter le compose pour la prod)
NODE_ENV=production docker compose -f docker-compose.prod.yml up -d
```

7.3 Option C — PaaS (Render, Railway, Fly.io)

1. **Backend** : déployer comme service Node.js
   - Build command : `npm ci --omit=dev`
   - Start command : `node src/index.js`
   - Variables d'environnement : toutes celles du `.env`
2. **Frontend** : déployer comme site statique (Vercel, Netlify)
   - Build command : `npm run build`
   - Output directory : `dist`
   - Variable : `VITE_API_URL=https://api.votredomaine.fr/api`
3. **Base de données** : service managé (Supabase, Neon pour PostgreSQL ; MongoDB Atlas)

SECTION 08

Sécurité

8.1 Mesures implémentées

MESURE	DÉTAIL
Helmet.js	Headers HTTP sécurisés (CSP, HSTS, X-Frame-Options, etc.)
CORS	Origines restreintes via `FRONTEND_URLS`
Rate limiting	200 req/15min (général), 20 req/15min (auth)
JWT	Access token 15min + refresh token 7 jours
bcrypt	Hash des mots de passe, 12 salt rounds
express-validator	Validation de toutes les entrées utilisateur
Mots de passe	≥10 caractères, majuscule + minuscule + chiffre + caractère spécial
Reset tokens	crypto.randomBytes(32), expiration 1h, usage unique
RGPD	Consentement à l'inscription, export données (Art. 20), anonymisation (Art. 17)

8.2 Checklist production

- Changer le mot de passe admin par défaut
- Générer des secrets JWT forts (`openssl rand -hex 32`)
- Mots de passe PostgreSQL et MongoDB forts
- Définir `NODE_ENV=production`
- HTTPS obligatoire (redirection HTTP → HTTPS)
- CORS : uniquement le domaine de production
- SMTP : serveur email de production (pas MailHog)
- Désactiver les ports PostgreSQL/MongoDB exposés sur Internet
- Sauvegardes automatiques de la base de données

9.1 Vérifications fonctionnelles

TEST	ENDPOINT / ACTION	RÉSULTAT ATTENDU
Health API	`GET /api/health`	`200 { "status": "ok" }`
Page d'accueil	`/`	Affichage correct
Inscription	Créer un compte	Email de bienvenue reçu
Connexion	Login avec email/mdp	JWT retourné, dashboard accessible
Consulter menus	`GET /api/menus`	Liste des menus actifs
Passer commande	Commander un menu	Commande créée, email de confirmation
Reset password	Mot de passe oublié	Email avec lien reçu, MDP modifiable
Admin	Se connecter en admin	Dashboard admin accessible
CORS	Requête cross-origin non autorisée	Rejetée (403)

9.2 Vérifications techniques

```
# Vérifier que l'API répond
curl -s https://votredomaine.fr/api/health | jq

# Vérifier les headers de sécurité
curl -I https://votredomaine.fr

# Vérifier le SSL
openssl s_client -connect votredomaine.fr:443 -brief

# Vérifier les logs
docker logs vg_api --tail 50
# ou
pm2 logs vg-api --lines 50
```

SECTION 10

Maintenance et opérations

10.1 Logs

```
# Docker
docker logs vg_api -f          # API (temps réel)
docker logs vg_postgres -f     # PostgreSQL

# PM2 (production VPS)
pm2 logs vg-api
```

L'API utilise **Pino** comme logger (JSON structuré en production, format lisible en développement).

10.2 Sauvegardes


```
docker exec vg_postgres pg_dump -U vitegourmand vitegourmand > backup_$(date +%Y%m%d).sql

# Restaurer
cat backup_20260307.sql | docker exec -i vg_postgres psql -U vitegourmand -d vitegourmand

# Backup MongoDB
docker exec vg_mongo mongodump --out /data/backup   --authenticationDatabase admin   -u vitegourmand -p <pa
```

10.3 Mises à jour

```
# Récupérer les dernières modifications
git pull origin main

# Reconstruire et relancer (Docker)
docker compose up --build -d

# Ou en production VPS
cd /var/www/vite-gourmand
git pull
cd backend && npm ci --omit=dev
cd ../frontend && npm ci && npm run build
pm2 restart vg-api
```

10.4 Commandes utiles

```
# Accéder à la DB PostgreSQL
docker exec -it vg_postgres psql -U vitegourmand -d vitegourmand

# Accéder au shell MongoDB
docker exec -it vg_mongo mongosh -u vitegourmand -p <password>

# Recréer les node_modules après ajout de dépendance
docker exec vg_api npm install && docker restart vg_api
docker exec vg_frontend npm install && docker restart vg_frontend

# Vérifier l'état des conteneurs
docker compose ps

# Voir l'utilisation des ressources
docker stats
```